

Guard App — WebSocket & Real-time Integration

Real-time communication is at the core of the guard app. Guards receive booking requests, status changes, and SOS alerts instantly via a persistent WebSocket connection. This document covers the full WebSocket service, message protocol, notification handling, and edge cases.

1. Connection Lifecycle

The guard connects to the WebSocket server when they go **online** and disconnects when they go **offline** or the session ends.

Guard toggles Online



```
guardWebSocketService.connect(guardId)
ws://api.bsecure.in/ws/guard/{guard_id}?token=<JWT>
```



(connection established)

Guard is reachable for booking requests



Receive: booking.new_request
Receive: booking.request_cancelled
Receive: booking.status_changed
Receive: sos.alert
Receive: admin.message
Send: location.update (every 10s)
Send: booking.accepted / declined
Send: booking.arrived / started / completed



Guard toggles Offline



```
guardWebSocketService.disconnect()
```

2. Message Protocol

All messages follow a typed envelope:

```
interface WSMMessage<T = unknown> {  
  type: string;      // e.g. "booking.new_request"  
  payload: T;  
}
```

Messages Received by Guard

Type	Description	Action
booking.new_request	New booking available near guard	Show incoming request UI, play sound, vibrate
booking.request_cancelled	User cancelled before guard responded	Dismiss incoming request UI
booking.status_changed	Status update on active booking	Update active booking state
sos.alert	User triggered SOS during active booking	Show full-screen SOS alert immediately
admin.message	Platform notification from admin	Show in-app notification banner

Messages Sent by Guard

Type	Description	When
location.update	Guard's current GPS coords	Every 10s during active booking
booking.accepted	Guard accepted a request	After successful accept API call
booking.declined	Guard declined / timed out	On decline or 30s timeout
booking.arrived	Guard arrived at pickup	When "Arrived" step is tapped
booking.started	Service started	When "Start Service" step is tapped
booking.completed	Service completed	

Type	Description	When
		When “Complete” step is confirmed

3. Guard WebSocket Service

```
// services/websocketService.ts
import { WS_URL } from '@constants/config';

type MessageHandler<T = any> = (payload: T) => void;

interface WSMMessage<T = any> {
  type: string;
  payload: T;
}

export class GuardWebSocketService {
  private ws: WebSocket | null = null;
  private guardId: string | null = null;
  private token: string | null = null;
  private listeners: Map<string, Set<MessageHandler>> = new Map();
  private reconnectAttempts = 0;
  private maxReconnectAttempts = 10;
  private reconnectDelay = 1000; // ms, doubles each attempt
  private pingInterval: ReturnType<typeof setInterval> | null = null;
  private shouldReconnect = true;

  connect(guardId: string, authToken: string): void {
    this.guardId = guardId;
    this.token = authToken;
    this.shouldReconnect = true;
    this._connect();
  }

  private _connect(): void {
    if (this.ws && this.ws.readyState === WebSocket.OPEN) return;

    const url = `${WS_URL}/ws/guard/${this.guardId}?token=${this.token}`;

    this.ws = new WebSocket(url);

    this.ws.onopen = () => {
      console.log('[WS Guard] Connected');
      this.reconnectAttempts = 0;
      this.reconnectDelay = 1000;
      this._startPing();
      this._emit('connection.established', {});
    };
  }
}
```

```

};

this.ws.onmessage = (event) => {
  try {
    const message: WSMMessage = JSON.parse(event.data);
    this._emit(message.type, message.payload);
  } catch (err) {
    console.warn('[WS Guard] Failed to parse message:',
      event.data);
  }
};

this.ws.onerror = (error) => {
  console.error('[WS Guard] Error:', error);
};

this.ws.onclose = (event) => {
  console.log('[WS Guard] Closed:', event.code, event.reason);
  this._stopPing();
  this._emit('connection.closed', { code: event.code });

  if (this.shouldReconnect && this.reconnectAttempts <
    this.maxReconnectAttempts) {
    this._scheduleReconnect();
  }
};
}

disconnect(): void {
  this.shouldReconnect = false;
  this._stopPing();
  if (this.ws) {
    this.ws.close(1000, 'Guard went offline');
    this.ws = null;
  }
  this.guardId = null;
  this.token = null;
}

send<T>(message: WSMMessage<T>): void {
  if (this.ws?.readyState === WebSocket.OPEN) {
    this.ws.send(JSON.stringify(message));
  } else {
    console.warn('[WS Guard] Cannot send – not connected.
      Type:', message.type);
  }
}

on<T>(type: string, handler: MessageHandler<T>): void {
  if (!this.listeners.has(type)) {
    this.listeners.set(type, new Set());
  }
  this.listeners.get(type)!.add(handler as MessageHandler);
}

```

```

}

off<T>(type: string, handler: MessageHandler<T>): void {
    this.listeners.get(type)?.delete(handler as MessageHandler);
}

get isConnected(): boolean {
    return this.ws?.readyState === WebSocket.OPEN;
}

// — Typed send helpers

```

```

sendLocationUpdate(
    lat: number,
    lon: number,
    heading: number | null,
    speed: number | null,
    bookingId: string
): void {
    this.send({
        type: 'location.update',
        payload: { lat, lon, heading, speed, booking_id:
            bookingId },
    });
}

sendBookingAccepted(bookingId: string): void {
    this.send({ type: 'booking.accepted', payload: { booking_id:
        bookingId } });
}

sendBookingDeclined(bookingId: string, reason: 'guard_declined'
    | 'timeout'): void {
    this.send({ type: 'booking.declined', payload: { booking_id:
        bookingId, reason } });
}

sendArrived(bookingId: string): void {
    this.send({ type: 'booking.arrived', payload: { booking_id:
        bookingId } });
}

sendStarted(bookingId: string): void {
    this.send({ type: 'booking.started', payload: { booking_id:
        bookingId } });
}

sendCompleted(bookingId: string): void {
    this.send({ type: 'booking.completed', payload: { booking_id:
        bookingId } });
}

```

// — Internal helpers

```
private _emit(type: string, payload: unknown): void {
  this.listeners.get(type)?.forEach((handler) => {
    try {
      handler(payload);
    } catch (err) {
      console.error('[WS Guard] Handler error for', type, err);
    }
  });
}

private _startPing(): void {
  this.pingInterval = setInterval(() => {
    if (this.ws?.readyState === WebSocket.OPEN) {
      this.ws.send(JSON.stringify({ type: 'ping', payload: {} }));
    }
  }, 30_000); // Ping every 30s to keep connection alive
}

private _stopPing(): void {
  if (this.pingInterval) {
    clearInterval(this.pingInterval);
    this.pingInterval = null;
  }
}

private _scheduleReconnect(): void {
  this.reconnectAttempts++;
  const delay = Math.min(this.reconnectDelay * Math.pow(2,
    this.reconnectAttempts - 1), 30_000);
  console.log(`[WS Guard] Reconnecting in ${delay}ms (attempt $
    {this.reconnectAttempts})`);
  setTimeout(() => {
    if (this.shouldReconnect && this.guardId && this.token) {
      this._connect();
    }
  }, delay);
}

// Singleton export
export const guardWebSocketService = new GuardWebSocketService();
```

4. Message Type Definitions

// types/websocket.ts

```
export interface IncomingBookingRequest {
```

```
    id: string;
    user_id: string;
    user_name: string;
    user_photo: string | null;
    user_rating: number;
    booking_type: string;
    pickup_location: { lat: number; lon: number };
    pickup_address: string;
    distance_km: number;
    duration_hours: number;
    estimated_earnings: number;
    notes?: string;
    expires_at: string; // ISO datetime - 30s from sent time
}
```

```
export interface BookingRequestCancelledPayload {
    booking_id: string;
    reason: string;
}
```

```
export interface BookingStatusChangedPayload {
    booking_id: string;
    status: 'en_route' | 'arrived' | 'started' | 'completed' |
           'cancelled';
    changed_at: string;
}
```

```
export interface SOSAlertPayload {
    booking_id: string;
    user_id: string;
    user_name: string;
    user_phone: string;
    location: { lat: number; lon: number };
    triggered_at: string;
    message?: string;
}
```

```
export interface AdminMessagePayload {
    id: string;
    title: string;
    body: string;
    action_url?: string;
    sent_at: string;
}
```

```
export interface LocationUpdatePayload {
    lat: number;
    lon: number;
    heading: number | null;
    speed: number | null;
    booking_id: string;
}
```

5. Notification Sound + Vibration on New Request

When a new booking request arrives, the guard must be alerted even if their phone is on silent. Use expo-av for sound and the Vibration API for haptics.

```
// services/alertService.ts
import { Vibration, Platform } from 'react-native';
import { Audio } from 'expo-av';

let notificationSound: Audio.Sound | null = null;

export async function preloadNotificationSound(): Promise<void> {
  try {
    await Audio.setAudioModeAsync({
      allowsRecordingIOS: false,
      staysActiveInBackground: false,
      playsInSilentModeIOS: true,      // Play even on silent
      mode (iOS)
      shouldDuckAndroid: true,
      playThroughEarpieceAndroid: false,
    });

    const { sound } = await Audio.Sound.createAsync(
      require('@assets/sounds/incoming_request.mp3'),
      {
        shouldPlay: false,
        volume: 1.0,
        isLooping: false,
      }
    );
    notificationSound = sound;
  } catch (err) {
    console.warn('[Alert] Failed to preload sound:', err);
  }
}

export async function playIncomingRequestAlert(): Promise<void> {
  // 1. Vibration pattern: buzz-pause-buzz-pause-buzz
  const VIBRATE_PATTERN = [0, 500, 200, 500, 200, 500];
  Vibration.vibrate(VIBRATE_PATTERN);

  // 2. Play sound
  try {
    if (notificationSound) {
      await notificationSound.setPositionAsync(0);
      await notificationSound.playAsync();
    }
  } catch (err) {
    console.warn('[Alert] Failed to play sound:', err);
  }
}
```



```

}

export function stopIncomingRequestAlert(): void {
  Vibration.cancel();
  notificationSound?.stopAsync().catch(() => {});
}

export async function unloadNotificationSound(): Promise<void> {
  if (notificationSound) {
    await notificationSound.unloadAsync();
    notificationSound = null;
  }
}

```

6. SOS Alert — Full-Screen Handler

SOS alerts must interrupt whatever the guard is doing and show a full-screen, unmissable overlay. This is handled by listening to the `sos.alert` WebSocket message and immediately displaying a modal.

```

// hooks/useSOSAlert.ts
import { useEffect, useRef } from 'react';
import { Vibration, AppState } from 'react-native';
import { useRouter } from 'expo-router';
import { Audio } from 'expo-av';
import * as Notifications from 'expo-notifications';
import { guardWebSocketService } from '@services/websocketService';
import { useActiveBookingStore } from '@store/activeBookingStore';
import type { SOSAlertPayload } from '@types/websocket';

export function useSOSAlert() {
  const router = useRouter();
  const { activeBooking } = useActiveBookingStore();
  const sosAlertRef = useRef<Audio.Sound | null>(null);

  useEffect(() => {
    const handleSOSAlert = async (payload: SOSAlertPayload) => {
      // Only handle SOS for the current active booking
      if (activeBooking && payload.booking_id !== activeBooking.id) return;

      // Persistent vibration
      Vibration.vibrate([0, 1000, 300], true /* repeat */);

      // Play alarm sound
      try {
        const { sound } = await Audio.Sound.createAsync(
          require('@assets/sounds/sos_alarm.mp3'),

```

```

        { shouldPlay: true, volume: 1.0, isLooping: true }
    );
    sosAlertRef.current = sound;
} catch {}

// Send local notification (wakes device from lock screen)
await Notifications.scheduleNotificationAsync({
  content: {
    title: '🚨 SOS ALERT',
    body: `${payload.user_name} has triggered an emergency!`,
    sound: true,
    priority: Notifications.AndroidNotificationPriority.MAX,
    vibrate: [0, 500, 250, 500],
    data: { type: 'sos', booking_id: payload.booking_id },
  },
  trigger: null, // Immediate
});

// Navigate to SOS screen
router.push({
  pathname: '/booking/active',
  params: { sos: 'true', booking_id: payload.booking_id },
});

guardWebSocketService.on('sos.alert', handleSOSAlert);
return () => {
  guardWebSocketService.off('sos.alert', handleSOSAlert);
};
}, [activeBooking?.id]);

const dismissSOSAlert = async () => {
  Vibration.cancel();
  if (sosAlertRef.current) {
    await sosAlertRef.current.stopAsync();
    await sosAlertRef.current.unloadAsync();
    sosAlertRef.current = null;
  }
};

return { dismissSOSAlert };
}

```

7. Handling Simultaneous Booking Requests

It is possible (but rare) for a guard to receive a second request while already handling one, or for two guards to attempt accepting the same booking. The handling rules are:

Rule 1: Only one request handled at a time

```
// In useIncomingBookingRequest hook:
const handleNewRequest = (request: IncomingRequest) => {
  // If already handling a request, ignore new ones completely
  if (incomingRequest) {
    // Auto-decline the new one immediately
    guardWebSocketService.sendBookingDeclined(request.id,
      'guard_declined');
    return;
  }
  // Process the incoming request...
};
```

Rule 2: Race condition on accept (HTTP 409)

```
// In useAcceptBooking hook:
const acceptMutation = useMutation({
  mutationFn: () => bookingService.acceptBooking(bookingId),
  onError: (error: any) => {
    if (error?.response?.status === 409) {
      // Another guard accepted first – silently dismiss
      setIncomingRequest(null);
      Alert.alert('Booking Taken', 'This booking was already
        accepted by another guard.');
```

Rule 3: Request cancelled before guard responds

```
// In useIncomingBookingRequest hook:
guardWebSocketService.on('booking.request_cancelled', (data) => {
  if (incomingRequest?.id === data.booking_id) {
    clearDeclineTimer();
    stopIncomingRequestAlert();
    setIncomingRequest(null);
    // Navigate back if on request screen
  }
});
```

8. Connecting with Auth Token

The WebSocket requires a valid JWT passed as a query parameter. Retrieve the stored token before connecting:

```
// hooks/useGuardStatus.ts (connection logic)
import { getStoredToken } from '@lib/storage';

const handleGoOnline = async () => {
```

```

    const token = await getStoredToken();
    if (!token || !guard?.id) return;

    guardWebSocketService.connect(guard.id, token);
  };

  // lib/storage.ts
  import * as SecureStore from 'expo-secure-store';

  const TOKEN_KEY = 'bsecure_guard_token';

  export async function getStoredToken(): Promise<string | null> {
    return SecureStore.getItemAsync(TOKEN_KEY);
  }

  export async function storeToken(token: string): Promise<void> {
    await SecureStore.setItemAsync(TOKEN_KEY, token);
  }

  export async function removeToken(): Promise<void> {
    await SecureStore.deleteItemAsync(TOKEN_KEY);
  }

```

9. Connection Status Indicator

Show the guard a real-time WebSocket connection status:

```

// hooks/useWebSocketStatus.ts
import { useState, useEffect } from 'react';
import { guardWebSocketService } from '@services/websocketService';

type WSStatus = 'connected' | 'disconnected' | 'reconnecting';

export function useWebSocketStatus(): WSStatus {
  const [status, setStatus] = useState<WSStatus>(
    guardWebSocketService.isConnected ? 'connected' :
    'disconnected'
  );

  useEffect(() => {
    const onConnected = () => setStatus('connected');
    const onClosed = () => setStatus('reconnecting');

    guardWebSocketService.on('connection.established',
      onConnected);
    guardWebSocketService.on('connection.closed', onClosed);

    return () => {
      guardWebSocketService.off('connection.established',
        onConnected);
    };
  }, []);
}

```

```
    guardWebSocketService.off('connection.closed', onClose);
  };
}, []);

return status;
}
```

10. Admin Messages

Admin messages are low-priority notifications displayed as banners:

```
// In dashboard or root layout:
useEffect(() => {
  const handleAdminMessage = (payload: AdminMessagePayload) => {
    // Show a toast/banner using your toast library
    Toast.show({
      title: payload.title,
      description: payload.body,
      type: 'info',
      duration: 6000,
    });
  };

  guardWebSocketService.on('admin.message', handleAdminMessage);
  return () => guardWebSocketService.off('admin.message',
    handleAdminMessage);
}, []);
```

11. Full Integration Checklist

☐

guardWebSocketService.connect(guardId, token) called when guard goes online

☐

guardWebSocketService.disconnect() called when guard goes offline or logs out

☐

booking.new_request listener registered in useIncomingBookingRequest

☐

sos.alert listener registered globally (root layout or useS0SAlert)

☐

Notification sound preloaded via preloadNotificationSound() on app start

☐

booking.accepted sent via WebSocket after successful HTTP accept



booking.declined sent on manual decline **and** on 30s timeout



location.update sent via WebSocket when foreground, via HTTP fetch when background



Reconnect logic handles app backgrounding (WS may be killed by OS after a few minutes)